Quest 9: The Sulphurprise

Lord of the Wings · a plant whose branches keep splitting in three

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: a plant starts as one branch that grows one element per second; whenever a branch reaches the odd length *growth*, it splits into two halves around a branching point and sprouts a third, new branch. How many seconds until the plant reaches its *limit*?
Input: one plant per line: `id=…, growth=…, limit=…`.
Output: the sum of the stopping times of all plants.
What changes between parts: Part I limits the total **size** (all elements). Part II limits the number of **leaves** (branch tips). Part III is Part II with huge numbers, so you can't simulate second by second any more.

1. The story

While the Ents grow their army, the team plans to sneak into Nulldor through the sewers and blow it up from inside. The weapon is a plant that grows explosively when it touches sulphur, and Nulldor is full of sulphur. Before carrying it in, the hero wants to know exactly how fast each variety grows, first by size, then by number of growing tips, and finally for much bigger varieties.

2. Understanding the question

The rules

- After second 1 there is one branch, 1 element long. The seed itself never counts.
- Every second, **every** branch grows by one element, all at the same time.
- When a branch reaches exactly *growth* elements (always odd), it splits at that moment: the middle element becomes a **branching point**, the two sides become two branches of (*growth* − 1) / 2 each, and a **new branch**, already 1 element long, starts from the branching point.
- The new branch doesn't grow again in the same second. From the next second, all three grow, and each can split later.
- The **size** is all branch elements plus all branching points. A **leaf** is a growing tip: there is 1 at the start and each split adds exactly 1 (the new branch's tip). So leaves = 1 + number of splits. Note this is *not* the number of branches: a split turns one branch into three, but adds only one leaf.
- The time for a plant is the first second at the end of which the measured value is ≥ *limit*.

One split, slowly

```
growth = 5, one branch growing:

second 4:   ████            branch of 4 elements          size 4
second 5:   █████  ← reached 5, so it splits right now:
            ██ + ██         two halves of (5 − 1) / 2 = 2
            the middle one  becomes a branching point (counts 1, not a branch)
            █               a new branch, already 1 long
                                                          size 2 + 2 + 1 + 1 = 6
second 6:   ██ ██ ██        all three grow by 1           size 3 + 3 + 2 + 1 = 9
```

A split adds two elements at once: the branching point and the new 1-element branch (the old branch's elements are only re-labelled). That's why the size jumps from 4 to 6 in the example above, and from 4 to 6 in the puzzle's plant 2 table.

Extra examples of my own (run through copies of your code)

growth	limit	Part I time	Why
3	1	1	The plant already has size 1 after the first second, so it stops immediately.
3	4	3	Size goes 1, 2, 4: the split in second 3 jumps from 2 straight past 3 to 4, which reaches the limit exactly.
5	6	5	Sizes 1, 2, 3, 4, 6. The limit is reached in the split second itself.

Shared code: reading the notes

```ts
import { readFileSync } from "node:fs";

const isSample = process.argv.includes("--sample");
const file = isSample ? "sample1.txt" : "input1.txt";
const INPUTS: string[] = readFileSync(file, "utf8").trim().split("\n");

const plants: {
  id: number;
  growth: number;
  limit: number;
}[] = [];

INPUTS.forEach((i) => {
  const [id = "", growth = "", limit = ""] = i
    .split(", ")
    .map((s) => s.split("=").slice(1).join("="));

  plants.push({
    id: Number(id),
    growth: Number(growth),
    limit: Number(limit),
  });
});
```

q9/part1.ts, lines 1–23

Lines	What happens
3–5	`--sample` picks the sample file, otherwise the real input. One string per line.
13–16	Split each line at `", "` into three `key=value` pieces, and keep what is after the `=`. The `slice(1).join("=")` would even keep a value that contains an `=` itself. The keys are ignored, so the code relies on the order `id, growth, limit`.
18–22	Store each plant as numbers. Part III uses `BigInt` here instead (lines 9–10 and 20–21 there).

3. Part I: limit the size

The question

Count every element of every branch plus every branching point. Find the first second where that total is ≥ *limit*, and add up these times.

Worked examples

Plant	growth	limit	Sizes, second by second	Time
1	3	10	1, 2, 4, 7, **13**	5
2	5	20	1, 2, 3, 4, 6, 9, 12, 17, **25**	9
3	7	30	1, 2, 3, 4, 5, 6, 8, 11, 14, 17, 22, 29, **37**	13
			sum	**27**

Step-by-step solution

1. Read each plant's *growth* and *limit*.
2. Start with one branch of length 1 and no branching points, at second 1.
3. While (sum of branch lengths + branching points) is below the limit: grow every existing branch by 1; any branch that reaches *growth* becomes a half, and a second half plus a new 1-branch are added (not grown this second); count a branching point. Add a second.
4. The second when the loop stops is the plant's time; add up all plants.

My solution

```ts
const getTime = (plant: (typeof plants)[number]): number => {
  let second = 1;
  let elements: number[] = [1];
  let branches = 0;

  while (
    elements.reduce((prev, curr) => prev + curr, 0) + branches <
    plant.limit
  ) {
    elements.forEach((e, idx) => {
      elements[idx] = e + 1;

      if (elements[idx] === plant.growth) {
        branches++;

        const splitted = (plant.growth - 1) / 2;
        elements[idx] = splitted;
        elements.push(splitted);
        elements.push(1);
      }
    });

    second++;
  }

  return second;
};

let totalTime = 0;

plants.forEach((p) => {
  totalTime += getTime(p);
});

console.log(totalTime);
```

q9/part1.ts, lines 25–59

Lines	What happens
26–28	The state after second 1: `elements` holds the length of every branch (one branch of length 1), and `branches` counts the **branching points** (zero so far). The name is a bit misleading: it counts splits, not branches.
30–33	Keep going while size = (sum of branch lengths) + (branching points) is below the limit. The sum is recomputed every second.
34–35	Grow every branch by one.
37–44	If a branch just reached `growth`: one more branching point, the branch itself becomes the first half, and two new entries are added at the end: the second half and the new 1-element branch.
47	One second has passed.
50	When the loop stops, `second` is the first second whose size reached the limit.
53–59	Add up the times of all plants and print.

Trace: plant 1 (growth 3, limit 10), from an instrumented copy

`second`	`elements` after the second	`branches`	size	≥ 10?
1	[1]	0	1	no
2	[2]	0	2	no
3	[1, 1, 1]	1	4	no
4	[2, 2, 2]	1	7	no
5	[1, 1, 1, 1, 1, 1, 1, 1, 1]	4	13	**yes**: return 5

And plant 2 (growth 5): in second 8 the array is `[2, 2, 4, 2, 1, 2, 1]` with 3 branching points, size 14 + 3 = 17, just like the puzzle's table.

> **Why the new branches don't grow in the split second: this depends on `forEach`.** Lines 42–43 push onto the array that line 34 is looping over. `forEach` decides how many items to visit *before* it starts, so the pushed branches are skipped until the next second, exactly as the rules say. If you rewrote it as `for (let i = 0; i < elements.length; i++)`, the loop would also grow the new entries immediately. I tried that in a copy: the sample times become 4, 8 and 11 instead of 5, 9 and 13. It works, but for a reason that's easy to break.

4. Part II: limit the leaves

The question

Same growth, but now the limit is on the number of **leaves** (branch tips). Every split adds one leaf. Stop at the first second with leaves ≥ *limit*.

Worked examples

Plant	growth	limit	Leaves, second by second	Time
1	3	10	1, 1, 2, 2, 5, 5, **14**	7
2	5	20	1, 1, 1, 1, 2, 2, 2, 4, 5, 5, 9, 13, 14, **22**	14
3	7	30	1 ×6, 2 ×4, 4, 4, 5, 5, 9, 9, 13, 13, 22, 22, **34**	21
			sum	**42**

growth	limit	Part II time	Why
9	2	9	The first split happens when the first branch reaches 9, in second 9.
3	1	1	One leaf from the start.
3	5	5	Leaves 1, 1, 2, 2, 5: in second 5 three branches split at the same time, +3 at once.

Step-by-step solution

1. Do the same simulation as Part I.
2. Keep a leaf counter that starts at 1 and goes up by 1 for each split.
3. Stop when leaves ≥ limit instead of checking the size.

My solution

Lines 1–23 are the shared code (with `sample2.txt` / `input2.txt`).

```ts
const getTime = (plant: (typeof plants)[number]): number => {
  let second = 1;
  let elements: number[] = [1];
  let leaves = 1;
  let branches = 0;

  while (leaves < plant.limit) {
    elements.forEach((e, idx) => {
      elements[idx] = e + 1;

      if (elements[idx] === plant.growth) {
        branches++;
        leaves++;

        const splitted = (plant.growth - 1) / 2;
        elements[idx] = splitted;
        elements.push(splitted);
        elements.push(1);
      }
    });

    second++;
  }

  return second;
};

let totalTime = 0;

plants.forEach((p) => {
  totalTime += getTime(p);
});

console.log(totalTime);
```

q9/part2.ts, lines 25–58 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
28	New counter `leaves`, 1 after the first second.
31	The loop now checks the leaf count instead of the size. No more summing the array every second.
37	Every split adds a leaf.
29, 36	`branches` is still counted, but nothing reads it any more. It's left over from Part I and could go.

Trace: plant 1 (growth 3, limit 10)

Second	1	2	3	4	5	6	7
`leaves`	1	1	2	2	5	5	14

The leaf count only changes in the seconds when branches split (3, 5, 7). For growth 3 every branch splits every 2 seconds, so the count goes 1 → 2 → 5 → 14 (each time ×3 − 1). When the loop stops, the array holds 27 branches. Plant 3 ends with 67 branches in the array after 21 seconds, so the array has about twice as many entries as there are leaves (27 vs 14 for plant 1).

> **Cost:** one step per second, and each step touches every branch, so the work is about (seconds × branches). That's fine here (the real input runs in about 0.1 s), but it's exactly what Part III makes impossible: with a leaf limit in the billions, the array alone would need billions of entries.

5. Part III: the same rules, at scale

The question

Exactly the rules of Part II, with much larger *growth* and *limit* values. The puzzle gives no new example: on the sample, Part III must give the Part II answer, 42 (and your code does).

The two key ideas

1. **Branches of the same length behave the same.** You don't need each branch, only how many branches have each length: a map *length* → *count*. Lengths are always between 1 and *growth* − 1, so the map stays small, however many branches there are.
2. **Nothing interesting happens between splits.** Leaves only change when something splits, and the next split is always the longest branch reaching *growth*, in exactly *growth* − *longest* seconds. So jump straight there and add that jump to every length.

Step-by-step solution

1. Store how many branches have each length, not every branch.
2. Find the longest length; the next split is *growth* − longest seconds away. Jump the clock there.
3. Add the jump to every length. Groups that reach *growth* add their count to the leaves and turn into 2 × count halves plus count new 1-branches.
4. Repeat until leaves ≥ limit; use `bigint` for everything, and add up the times.

My solution

Lines 1–23 are the shared code, but with `bigint` for `growth` and `limit` (lines 9–10, 20–21).

```ts
const getTime = (plant: (typeof plants)[number]): bigint => {
  let groups = new Map<bigint, bigint>([[1n, 1n]]);
  let second = 1n;
  let leaves = 1n;

  const half = (plant.growth - 1n) / 2n;

  while (leaves < plant.limit) {
    let longest = 0n;

    for (const len of groups.keys()) if (len > longest) longest = len;

    const jump = plant.growth - longest;
    second += jump;

    const next = new Map<bigint, bigint>();
    const add = (len: bigint, count: bigint) =>
      next.set(len, (next.get(len) ?? 0n) + count);

    for (const [len, count] of groups) {
      const grown = len + jump;

      if (grown === plant.growth) {
        leaves += count;
        add(half, 2n * count);
        add(1n, count);
      } else {
        add(grown, count);
      }
    }

    groups = next;
  }

  return second;
};

let totalTime = 0n;

plants.forEach((p) => {
  totalTime += getTime(p);
});

console.log(totalTime.toString());
```

q9/part3.ts, lines 25–68 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
26	`groups`: length → number of branches with that length. Start: one branch of length 1.
27–30	Time and leaves as in Part II, now as `bigint`. `half` is the length of each half after a split.
33–35	Find the longest branch length in the map.
37–38	That branch splits in `growth − longest` seconds. Jump the clock forward by that much.
40–42	Build the next map. `add` adds to a count, so different groups that end up at the same length are merged.
44–54	Every group grows by `jump`. The groups that reached `growth` split: each of their `count` branches makes one new leaf, two halves and one new 1-branch. Everything else just gets longer.
56	Swap in the new map and check the leaf limit again.
62–68	Add up as `bigint`, then print it as a string, which keeps every digit exact.

Trace: plant 2 (growth 5, limit 20), from an instrumented copy

longest	jump	second	groups after the jump (length×count)	leaves
		1	1×1	1
1	4	5	1×1, 2×2	2
2	3	8	1×2, 2×4, 4×1	4
4	1	9	1×1, 2×4, 3×4	5
3	2	11	1×4, 2×8, 3×1, 4×4	9
4	1	12	1×4, 2×12, 3×8, 4×1	13
4	1	13	1×1, 2×6, 3×12, 4×8	14
4	1	14	1×8, 2×17, 3×6, 4×12	**22**

Seven jumps instead of 13 seconds, and the leaf counts at seconds 5, 8, 9, 11, 12, 13 and 14 are exactly the puzzle's. Plant 1 (growth 3) is even simpler: everything always has length 1, so each jump is 2 seconds and the map is just `1×3`, `1×9`, `1×27`.

A bigger test of my own, growth 1001 and a leaf limit of 1,000,000: the first jump is 1000 seconds (to second 1001), and after that the map never holds more than about a dozen lengths, while the counts climb into the tens of thousands. On your real input, all plants together need about 15,000 jumps, the map never holds more than 38 lengths, and the run takes under 0.1 s.

> **Why it's safe to jump:** no branch can reach *growth* before the longest one does, because all branches grow at the same speed. And between two jumps the leaf count doesn't change, so the limit can only be reached at a jump. The puzzle's "end of the first second when leaves ≥ limit" is therefore always one of the jump times.

> **Printing the total:** line 68 prints the `bigint` as a string, so the result stays exact however large it gets. (Turning it into a normal JavaScript number first would only be exact up to $2^{53} \approx 9.0 \times 10^{15}$.)
> **Also:** the "find the longest" scan on line 35 and the full map rebuild each jump are fine because the map is tiny. If it ever had many lengths, you could keep the lengths sorted, but there's no need here.

6. Summary

Part	Limit on	State kept	Time step	Sample
I	size (elements + branching points)	array of every branch length	1 second	27
II	leaves (1 + number of splits)	same array, plus a leaf counter	1 second	42
III	leaves, huge values	map length → count (`bigint`)	jump to the next split	42

Key ideas: things that behave the same can be **grouped and counted** instead of stored one by one (the same trick as the classic "lanternfish" style puzzles), and when nothing changes between events, **jump from event to event** instead of ticking every second.